

Углубленный Python

Лекция 5

Метaprogramмирование Стандартная библиотека

Кандауров Геннадий



education

Напоминание отметиться на портале

+ ОСТАВИТЬ ОТЗЫВ

vk образование

БлогиЛюдиПрограммаВакансииРасписание

Q<

VK

Техно

Открыт приём заявок!

чт, 8 сентября

Нет занятий

пт, 9 сентября

18:00 Углубленный Py... с3
Введение в Python, основные
понятия, тестирование
Г. Кандауров

сб, 10 сентября

Нет занятий

вс, 11 сентября

Нет занятий

пн, 12 сентября

Нет занятий

Углубленный Python

↓ 0 ↑

Блог для материалов по курсу "Углубленный Python"

57 читателей, 2 топика

Подписаться

Создать топик

Поиск по авторам, заголовку и тексту топика...

Найти

Добро пожаловать на курс!

Углубленный Python

Изменить

Удалить

Всем привет и добро пожаловать на курс по углубленному изучению Python!

Прямой эфир

МоиВсе

Геннадий Кандауров час назад
Углубленный Python → Добро пожаловать
на курс! 0

Екатерина Черкасова 7 дней назад
Стажировка → Приглашаем мобильных,
фронтенд- и бэкэнд-разработчиков на
Weekend Offer! 0

Дарья Вовченко 9 дней назад
Углубленный Python → Добро пожаловать
в образовательные проекты VK
Образование! 0

Дарья Вовченко 9 дней назад
Разработка веб-сервисов на

Квиз про прошлой лекции



Содержание занятия

1. Метaproгpаммиpование
2. Dataclasses, Enums
3. Pегуляpные выражения (re)

Классы



Классы: магические методы

```
object.__new__(cls[, ...])
```

Статический метод, создает новый экземпляр класса.

После создание экземпляра вызывается (уже у экземпляра) метод `__init__`.

`__init__` ничего не должен возвращать (кроме `None`), иначе - `TypeError`

```
class Singleton:
```

```
    _instance = None
```

```
    def __new__(cls, *args, **kwargs):
```

```
        if cls._instance is None:
```

```
            cls._instance = super().__new__(cls, *args, **kwargs)
```

```
        return cls._instance
```

Классы: `__init_subclass__`

```
class Timing:
    def __init__(self, start, end):
        self.start = start
        self.end = end

    @classmethod
    def __init_subclass__(cls, **kwargs):
        print("INIT subclass", cls, kwargs)
```

```
class MinuteTiming(Timing):
    def duration(self):
        print("MinuteTiming.duration")
        seconds = super().duration()
        return seconds / 60
```

Метаклассы

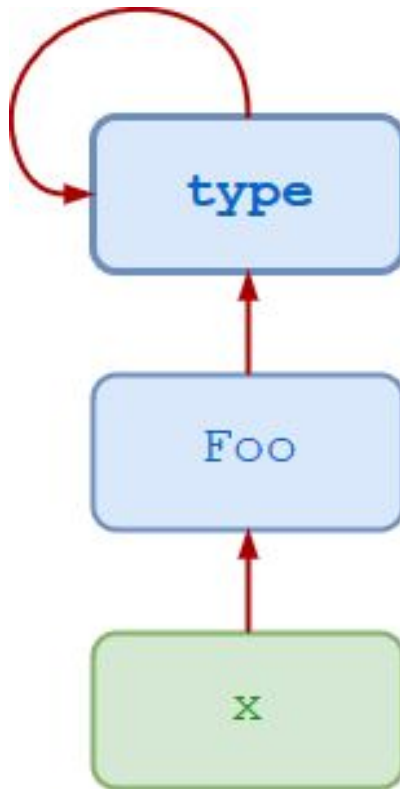
Классы, экземпляры которых
являются классами



Метаклассы: type

```
class Foo:  
    pass
```

```
x = Foo()
```



Метаклассы: `type`

Новые классы создаются с помощью вызова

`type(<name>, <bases>, <classdict>)`

`name` – имя класса (`__name__`)

`bases` – базовые классы (`__bases__`)

`classdict` – namespace класса (`__dict__`)

`MyClass = type("MyClass", (), {})`

Метаклассы: type

```
>>> Bar = type('Bar', (Foo,), dict(attr=100))
```

```
>>> x = Bar()
```

```
>>> x.attr
```

```
100
```

```
>>> x.__class__
```

```
<class '__main__.Bar'>
```

```
>>> x.__class__.__bases__
```

```
(<class '__main__.Foo'>,)
```

```
>>> class Bar(Foo):
```

```
...     attr = 100
```

```
...
```

```
>>> x = Bar()
```

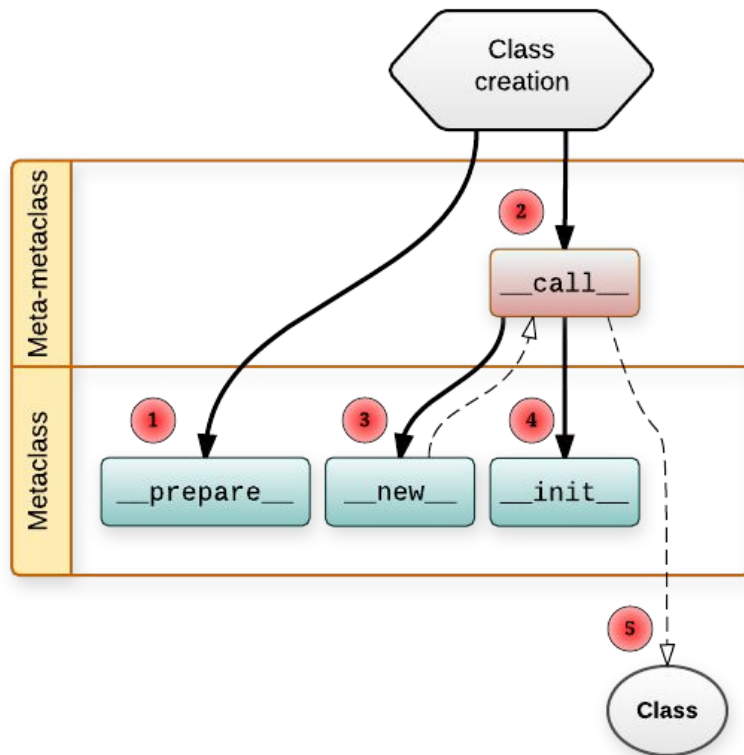
```
>>> x.attr
```

```
100
```

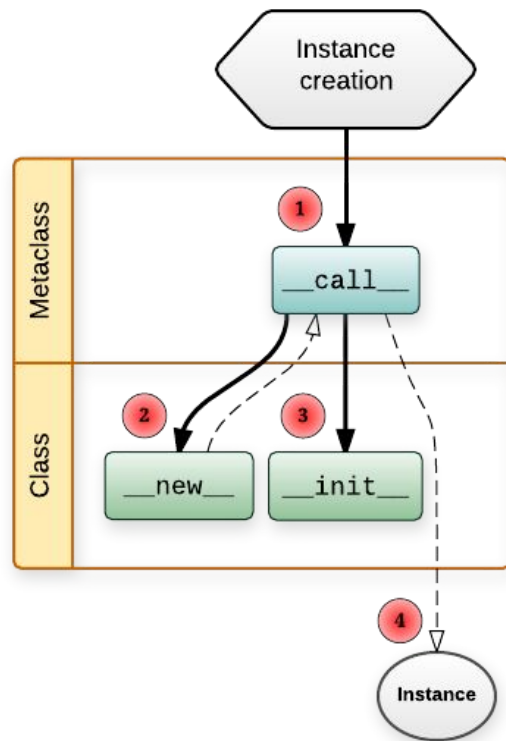
```
>>> x.__class__.__bases__
```

```
(<class '__main__.Foo'>,)
```

Метаклассы: создание класса



Метаклассы



Метаклассы: создание класса

- определяются базовые классы
- определяется метакласс
- подготавливается namespace класса (`__prepare__`)
- выполняется тело класса
- создается класс (`__new__`, `__init__`)

Метаклассы

```
class AMeta(type):  
    def __new__(mcs, name, bases, classdict, **kwargs):  
        cls = super().__new__(mcs, name, bases, classdict)  
        print("Meta __new__", cls)  
        return cls  
  
    def __init__(cls, name, bases, classdict, **kwargs):  
        super().__init__(name, bases, classdict, **kwargs)  
  
    def __call__(cls, *args, **kwargs):  
        return super().__call__(*args, **kwargs)  
  
    @classmethod  
    def __prepare__(mcs, name, bases, **kwargs):  
        print("Meta __prepare__", **kwargs)  
        return {"b": 2, "a": 2}
```

Разное



collections.namedtuple

`namedtuple(typename, field_names, *, rename=False, defaults=None, module=None)`

```
>>> Point = collections.namedtuple("Point", ["x", "y"])
```

```
>>> p = Point(11, y=22)  # p = (11, 22)
```

```
>>> p[0] + p[1]
```

```
33
```

```
>>> x, y = p
```

```
>>> x, y
```

```
(11, 22)
```

```
>>> p.x + p.y
```

```
33
```

```
>>> p._asdict()  # {'x': 1, 'y': 4}
```

Enum

```
from enum import Enum
```

```
class HttpStatus(enum.Enum):
```

```
    OK = 200
```

```
    NOT_FOUND = 404
```

```
    SERVER_ERROR = 500
```

```
print(HttpStatus.OK.value, HttpStatus.OK.name) # (200, 'OK')
```

```
status = HttpStatus(200)
```

```
if status is HttpStatus.OK:
```

```
    print("OK")
```

Dataclasses

```
import dataclasses
```

```
@dataclasses.dataclass
```

```
class Point:
```

```
    x: int
```

```
    y: int
```

```
    vector: list[int] = dataclasses.field(default_factory=list)
```

```
p = Point(10, 20)
```

```
print(p.x, p.y, p.vector)  # (10, 20, [])
```

```
@dataclasses.dataclass(slots=True)
```

```
class PointSlots:
```

```
    x: int
```

```
    y: int
```

```
p = PointSlots(10, 20)
```

```
print(p)  # PointSlots(x=10, y=20)
```

re: регулярные выражения

Регулярные выражения — это специальный язык шаблонов для поиска, извлечения и замены текста

- `re.search(pattern, string)`: ищет совпадение по всему тексту, возвращает первый объект совпадения или `None`.
- `re.match(pattern, string)`: ищет совпадение **только в самом начале** строки.
- `re.findall(pattern, string)`: возвращает список всех найденных совпадений в виде строк.
- `re.finditer(pattern, string)`: возвращает итератор по всем объектам совпадений (полезно для получения позиций).
- `re.sub(pattern, repl, string)`: заменяет все вхождения `pattern` на `repl` в строке.
- `re.split(pattern, string)`: разбивает строку по шаблону

Домашнее задание #04

- Система модерации сообщений
- Тесты
- Зеленый пайплайн (тесты, coverage, линтеры)

Напоминание отметиться на портале Vol 2

+ ОСТАВИТЬ ОТЗЫВ ПОСЛЕ ЛЕКЦИИ

vk образование

БлогиЛюдиПрограммаВакансииРасписание

Q<

VK

Техно

Открыт приём заявок!

чт, 8 сентября

Нет занятий

пт, 9 сентября

18:00 Углубленный Py... с3
Введение в Python, основные
понятия, тестирование
Г. Кандауров

сб, 10 сентября

Нет занятий

вс, 11 сентября

Нет занятий

пн, 12 сентября

Нет занятий

Углубленный Python

↓ 0 ↑

Блог для материалов по курсу "Углубленный Python"

57 читателей, 2 топика

Подписаться

Создать топик

Поиск по авторам, заголовку и тексту топика...

Найти

Добро пожаловать на курс!

Углубленный Python

Изменить

Удалить

Всем привет и добро пожаловать на курс по углубленному изучению Python!

Прямой эфир

МоиВсе

Геннадий Кандауров час назад
Углубленный Python → Добро пожаловать
на курс! 0

Екатерина Черкасова 7 дней назад
Стажировка → Приглашаем мобильных,
фронтенд- и бэкэнд-разработчиков на
Weekend Offer! 0

Дарья Вовченко 9 дней назад
Углубленный Python → Добро пожаловать
в образовательные проекты VK
Образование! 0

Дарья Вовченко 9 дней назад
Разработка веб-сервисов на

Спасибо за
внимание

